COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate route for games

04

Native programming with C++

Module 11 / expanded Volume 03

Four lessons, worked examples, full practical requirements and 20 lesson questions.

PACKAGE 04 / EXPANDED VOLUME 03 / SEPTEMBER 28, 2026
Independent study, implementation, evidence and review.

Contents

11.1 — Compilation, linking and build systems 3

11.2 — Integer, floating-point and object representations 7

11.3 — Values, references, pointers and lifetimes 10

11.4 — Standard containers, iterators and error handling 13

Continuity with the original program 16

Primary reference directory 16

Reading by topic 18

Module 11 — Native programming with C++

Prerequisites: Volume 01, Modules 06-07; do not skip the ownership module that follows.

Project: Port the deterministic grid simulation to C++20, with a command-line build, automated tests, warnings enabled and an explicit account of representation differences from Python.

Primary references: R11, R12, R15. Four lessons,20 expanded quiz questions and a module project that contributes to GateC.

11.1 — Compilation, linking and build systems

Learning outcome: Explain how source code becomes a runnable native program.

From a Python script to a native program

A source file is not an executable. In this bundle, CMake describes targets and dependencies; a generator produces build instructions; a compiler translates each C++ translation unit; a linker combines the resulting object files and required libraries. The operating system then loads the executable. A successful compile is evidence about one stage, not proof that the whole program is correct. A game can build, launch and still update the wrong entity.

A translation unit is approximately a source file after preprocessing, including the text brought in through headers. A header normally exposes declarations that several translation units need. The definition of an ordinary non-inline function belongs in one source file. Include guards prevent duplicate inclusion within a translation unit; they do not magically permit multiple ordinary function definitions across different translation units. Later you will meet the permitted multiple definitions of inline functions and templates.

Keep four questions separate: Can the compiler find the declaration? Does a matching definition exist? Did the linker receive the object or library containing it? Does the program satisfy the runtime contract? This separation turns a screen of diagnostics into a sequence of testable hypotheses. [R11: SF; R12]

Reading the smallest useful C++ program

The included example declares `int next_tick(int current);`, defines it in another file, and calls it from `main`. The semicolon terminates a declaration or statement. Braces define a body or scope. The return type precedes the function name; argument types are explicit. `#include` is a preprocessing directive, not a runtime function call. `std::cout` names the output object in the standard namespace; << streams a value into it.

Use a function's declaration as a contract, not merely a compiler appeasement. Our next-tick function accepts an integer below the maximum representable value and rejects the upper boundary before adding. A function named `next_tick` that silently wraps time would introduce an observable simulation rule, not a harmless implementation detail. Tests express that choice.

The `main` function returns zero on successful completion. Our test harness returns nonzero when a check fails, and CTest uses that status. Do not use an ordinary C `assert` as the only test mechanism: optimized builds may define `NDEBUG` and remove it. The included CHECK helper remains active in both Debug and Release configurations.

Targets, configuration and reproducibility

A CMake target gathers source files, include directories, language requirements and link dependencies under one name. Prefer target-scoped settings to machine-wide flags. The bundle requires C++20 explicitly and disables compiler extensions for the teaching examples. That is a selected baseline, not a claim about the newest language or complete compiler feature support.

An out-of-source build separates the authored files under `code/` from generated products under `build/`. A clean build can be reproduced without relying on yesterday's object files. Configure once, build the selected targets, then run tests. Reconfigure when toolchain or build options change. Never compare timings from an instrumented Debug build with an optimized uninstrumented Release build and attribute the entire difference to an algorithm.

Record the compiler identity, compiler version, standard-library implementation when known, CMake version, configuration and command line. Multi-configuration generators select Debug or Release at build time; single-configuration generators commonly select through `CMAKE_BUILD_TYPE`. The getting-started guide supplies commands for both. [R12; P4-01]

Diagnose one stage at a time

A missing semicolon is ordinarily a compile-time syntax error. A call with no visible declaration is also a compilation problem. A visible declaration with an omitted implementation can compile but fail to link. Two conflicting definitions may produce a duplicate-symbol failure. A wrong signature in the implementation can accidentally define a different overload, leaving the intended one unresolved.

Read the first relevant diagnostic before fixing the later cascade. Reduce the failing case to one target and preserve the exact command and message in the lab journal. Do not cure a link error by including a `.cpp` file inside another `.cpp` file; add that source to the target. Do not put `using namespace std;` in a public header, because it imposes names on every includer.

The broken examples are intentionally stored as text fixtures outside the default build. The optional diagnostic script runs a syntax-failure and an unresolved-symbol experiment and checks that both are rejected. There is no requirement to execute undefined behavior to learn what an invalid program looks like.

Trace a two-file build

`tick.hpp` declares `next_tick`. `tick.cpp` includes that header and supplies the body. `lesson_11_1.cpp` includes the same header and calls the function. Compiling the main file alone knows the argument and result types, but its object file can still contain an unresolved reference. Linking both object files provides the implementation.

Predict three experiments before running them: remove the declaration from the header; restore it and omit `tick.cpp` from the link; restore both files. The expected outcomes are compilation failure, link failure and successful execution. Record the actual diagnostic wording on your compiler rather than memorizing one vendor's message.

For input 41 the result is 42. At the maximum integer the function throws before arithmetic. The test catches that error. A clean build therefore demonstrates both the translation path and one explicit domain boundary.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_11_1.cpp`. Pseudocode excerpts are not standalone C++ source.

```
// tick.hpp
#pragma once
int next_tick(int current);

// tick.cpp
#include "tick.hpp"
#include <limits>
#include <stdexcept>
int next_tick(int current) {
    if (current == std::numeric_limits<int>::max())
        throw std::overflow_error("tick limit");
    return current + 1;
}
```

Common failure modes

A header that happens to compile because another header included a dependency is fragile: include what the header itself uses. A stale executable can appear to run after a failed build; verify the build exit status. Compiler warnings are evidence to investigate, not clutter to disable globally.

Practical assignment

Original requirement: Build the included C++ starter, then split one function into a header and source file. Pass: reproduce and explain one compile error and one link error, then restore a warning-clean build.

1. Configure and build the reference target. Save the full toolchain fingerprint and identify each build stage.
2. Recreate the two-file target in the starter directory. Change the function name consistently, then introduce and repair one declaration/definition mismatch.
3. Use the diagnostic fixtures to distinguish a compile error from a link error. Keep both messages in the journal.
4. Add a boundary test and rebuild from a new build directory. Explain why the test remains active in Release.

Required cases: ordinary input 0 -> 1; ordinary input 41 -> 42; maximum int -> overflow error before addition; missing declaration -> compile rejection; missing definition -> link rejection.

Evidence to submit: Two source files and a header, reproducible commands, actual diagnostic transcripts, a warning-clean restored build, and a labeled compile/link/load diagram.

Independent transfer: A teammate supplies a declaration taking `long` but a definition taking `int`. Explain why the implementation may not satisfy the call, then repair the mismatch without a cast at the call site.

Assigned primary reading: R12: Getting Started and libraries; R11: Source Files.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. What task does the linker perform?
2. Can valid declarations alone supply every function implementation?
3. Why record build configuration with a benchmark?
4. A build reports failure, but an executable still exists. Is its output evidence that your edit works?
5. Why does an include guard not solve ordinary duplicate function definitions in two translation units?

11.2 — Integer, floating-point and object representations

Learning outcome: Choose native types with explicit range and precision assumptions.

Values are mathematical; representations are finite

A game rule might describe an unbounded score, but a native integer represents a finite set. Eight bits provide 2^8 = 256 patterns. The meaning assigned to those patterns depends on the type. `std::numeric_limits<T>` describes limits on the current implementation; `sizeof(T)` counts C++ bytes, and `CHAR_BIT` states the number of bits in one byte. Do not silently assume every target has the same `int`, pointer or alignment.

Use `std::size_t` for object sizes and container indices where the library requires it, but do not use unsigned arithmetic as a substitute for domain validation. Converting a negative index to an unsigned value does not make it valid. Exact-width types such as `std::uint32_t` are useful when the implementation provides them; a file format must still specify byte order and an encoding independent of native object layout.

The first experiment prints widths and limits on your toolchain. Write down an assumption before printing the answer. A mismatch is useful evidence, not a failed lesson. The representation report belongs in the simulation port's handoff. [R11: ES; R15]

Reject invalid arithmetic before performing it

For unsigned addition modulo 2^N, wraparound is defined by the language, but it may still be wrong for the application. Ordinary signed overflow is undefined behavior in C++. A check after `a + b` is too late if that sum overflowed. To add signed integers safely, check `a > max - b` when `b > 0`, and `a < min - b` when `b < 0`; only then perform the addition.

For a buffer of `count` elements of size `element_size`, first require `element_size != 0` where that is part of the contract. To cap allocation by `limit`, reject `count > limit / element_size`; then multiplication cannot exceed the chosen limit. The limit might be lower than the machine maximum because the engine has a resource budget.

Parsing and converting are separate steps. Parse a nonnegative decimal quantity into a sufficiently wide unsigned type with a checked parser, verify the entire input was consumed, apply the application limit and then convert. Inputs such as `-1`, `12junk`, empty text and an excessive number must not turn into successful lengths.

Floating-point behavior needs a contract

A floating-point number stores a sign, a scale and a finite-precision significand in common binary implementations. Many decimal fractions have no finite binary expansion, so an operation rounds. Precision is not the same as range: a type can represent enormous magnitudes while losing the ability to distinguish nearby small differences at those magnitudes.

Use exact equality when testing an exact discrete decision or a deliberately exact arithmetic case. For approximate numerical outputs, choose an application-specific combined check: absolute error <= absolute tolerance + relative tolerance times the larger magnitude. This is a test policy, not a universal equivalence relation suitable for sorting. NaN is unordered in ordinary comparisons; explicitly reject nonfinite values where the simulation contract requires finite numbers.

Do not promise bit-identical floating-point replay across arbitrary compilers, architectures and optimization flags. This volume's simple grid uses bounded integers to make the state rules easier to reproduce. Later physics work needs its own deterministic or tolerance-based comparison policy. [R15; P4-02]

An object in memory is not a file format

A structure may contain padding between fields and at its end. Padding, byte order, field width and alignment can differ between targets. Saving `sizeof(struct)` bytes also exposes implementation details that become incompatible when the structure changes. Pointer values have no useful portable meaning in a save file.

Specify a wire or disk record field by field. For this lesson, encode a 32-bit unsigned value as four octets in little-endian order. The low byte contains bits 0 through 7; the next contains bits 8 through 15, and so on. The receiving function reconstructs the value with shifts on an appropriately wide unsigned type. This is independent of the host's byte order.

Byte inspection is an observation exercise, not permission to reinterpret unrelated object types arbitrarily. Use facilities intended for object representation and respect their constraints. Our teaching encoder deals with values, not uninitialized padding. The later C bridge prints actual structure offsets and then deliberately avoids using that raw layout as the packet format.

A checked asset allocation

Suppose an asset permits at most 1,024 bytes and each record needs 24 bytes. Testing `count > 1024 / 24` rejects count 43, because integer division yields 42. Count 42 produces 1,008 bytes and is accepted. No potentially overflowing multiplication is needed to discover that 43 is invalid.

For a 32-bit record value `0x12345678`, the defined little-endian bytes are `78 56 34 12` in hexadecimal. Decode by OR-ing the four unsigned byte values shifted by 0, 8, 16 and 24 bits. The round trip checks the encoding, but a second test must compare the encoded bytes with the independently specified sequence: a mutually wrong encoder and decoder can still round-trip.

For an external length, check negative syntax, the parser's out-of-range result, trailing characters and the application budget. A cast belongs after these decisions, not before them.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_11_2.cpp`. Pseudocode excerpts are not standalone C++ source.

```
bool fits_budget(std::size_t count,
                 std::size_t element_size,
                 std::size_t byte_limit) {
    return element_size != 0 &&
```

```
            count <= byte_limit / element_size;
}
```

Common failure modes

Never make signed overflow occur and then inspect whether it wrapped. Do not use an arbitrary epsilon for every number. A successful native-layout round trip on one machine does not establish portable serialization.

Practical assignment

Original requirement: Print relevant sizes and numeric limits on the available compiler. Write checked conversions for a file length and tests for boundary values. Pass: do not trigger signed overflow as a test oracle; reject out-of-range input before converting.

1. Print numeric limits, sizes, byte width and selected alignment values. Mark which are observations rather than portable constants.
2. Implement checked decimal length parsing and pre-multiplication budget checks.
3. Write a field-based little-endian encoder/decoder and compare with independently written bytes.
4. Test numeric boundaries and document the difference between Python integers and the chosen native representation.

Required cases: 0 and exact application maximum accepted; negative text and trailing junk rejected; one above the application limit rejected; huge decimal value rejected by parser; 42 records of 24 bytes accepted under 1024; 43 rejected; 0x12345678 encodes to 78 56 34 12.

Evidence to submit: Representation report, boundary-test table, checked conversion implementation, specified encoding and a comparison with the earlier Python model.

Independent transfer: A texture claims width = 100,000 and height = 100,000. Design checks for width * height * bytes_per_pixel without overflowing during validation.

Assigned primary reading: R11: arithmetic and conversion guidance; R15: information encoding and arithmetic.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. How many distinct bit patterns fit in eight bits?
2. Is ordinary signed integer overflow defined to wrap in C++?
3. Why is writing raw struct bytes not a portable format?
4. Why is `if (a + b > max)` not a safe signed-overflow check?
5. Can a serializer and deserializer round-trip correctly yet disagree with the file specification?

11.3 — Values, references, pointers and lifetimes

Learning outcome: Track where an object lives and how long access remains valid.

Identity, copying and borrowing

A value parameter creates a separate parameter object. For a small position or entity identifier, copying often expresses the intended independence. A reference names an existing object; a pointer stores an address-like value that can be null when the interface permits it. Neither a raw pointer nor a reference communicates ownership automatically.

`const T&` lets a function borrow an existing object without copying and prevents mutation through that reference. It does not keep the object alive. It does not prevent another alias from changing the object. It also does not turn a type with hidden mutable state into a thread-safe object. A useful interface states not only what may be modified, but for how long access is retained.

Our damage calculation copies two small integers. Our read-only unit lookup borrows a container. A nullable pointer parameter has an explicit policy: null means no target and returns a not-found result, not an unchecked dereference. Choose parameter form from these semantics rather than from a blanket rule that pointers are fast or references are safe. [R11: F, I, R]

Storage duration is not the entire lifetime story

An automatic local object normally lives until its scope exits. A dynamically allocated object lives until its owner releases it. A container owns its elements, but a reallocation can end the old element lifetimes even while the container itself remains alive. Therefore a pointer's numerical address is not a sufficient proof that it designates the object you intended.

A function must not return a reference or pointer to a local variable that is about to be destroyed. Returning a value is often the simplest repair. Returning a view into caller-owned data can be valid when the caller retains the owner and performs no invalidating operation. Document that dependency explicitly.

Some C++ rules extend certain temporary lifetimes, but not every chain of references extends a temporary. For this course, avoid storing a borrowed reference to a temporary. A `std::string_view` or `std::span` owns no storage. Constructing a view from a short-lived owner and keeping the view is a common lifetime defect, even though the view object itself looks ordinary.

Draw an ownership and access diagram

For each important object, label one owner, zero or more borrowers, and the operations that end or relocate it. Draw arrows from borrowers to the owner. A unit vector owns its unit values. A lookup result can borrow one unit until the next invalidating mutation; an integer unit ID is a value that can be looked up again after mutation.

Stable identity is a separate design problem from stable address. An index remains an integer after vector growth, but erasing or reordering elements may make it refer to a different entity. A later engine can use generation-counted handles to detect recycled slots. Here, keep IDs unique and perform explicit lookup instead of pretending an index is a permanent entity identity.

The lab asks you to analyze a dangling-reference example as text. Do not execute the invalid access to see whether it crashes. Undefined behavior need not produce a crash and is not a reliable teaching oracle. Sanitizers add useful checks on paths that actually run, but an unreported defect is not a proof of lifetime safety. [P4-03; P4-04]

Bounds belong with the borrow

Passing a raw pointer to the first element without a length loses the range boundary. `std::span<const Unit>` carries a contiguous range and its size while remaining non-owning. The caller must still keep the elements alive. A span is not a memory-ownership system and ordinary indexing is not an automatic guarantee of checked bounds in the C++20 baseline.

When the function should consume only a subset, pass a bounded range or explicit start/end convention. Validate external indices before accessing elements. Keep the borrowed reference local to the operation rather than storing it in long-lived engine state. A clear lifetime usually does more for reliability than a clever pointer trick.

In the code example, a query returns an optional copied position rather than a reference. That choice spends a small copy to avoid exporting a lifetime obligation. For large assets, a borrowed view can be appropriate, but the interface must state the mutation and destruction events that invalidate it.

A lookup that does not export a dangling reference

A vector contains unit 7 at position (2,3). `position_of` scans a borrowed span and returns an optional Position value. Finding unit 7 copies (2,3) into the result. The caller can grow or destroy the vector without invalidating that copied result. A missing ID returns no value, not a fabricated position (0,0).

Compare a function returning `const Position&` from a local `Position p`. The local lifetime ends at return, so the reference is invalid immediately. The correct repair is not to make the reference `const`; it is to return a value or choose an owner whose lifetime actually covers use.

Draw the diagram twice: once for a copied result and once for a borrowed pointer into the vector. List the new obligations introduced by the borrowed version.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_11_3.cpp`. Pseudocode excerpts are not standalone C++ source.

```
std::optional<Position> position_of(
    std::span<const Unit> units, int id) {
    for (const Unit& u : units)
        if (u.id == id) return u.position;
    return std::nullopt;
}
```

Common failure modes

`const` is not a lifetime extension. A non-null pointer can still dangle. An index can remain in bounds yet name the wrong entity after an erase. A view copied by value still borrows its underlying storage.

Practical assignment

Original requirement: Implement functions taking values, const references and pointers with a documented null policy. Pass: explain each lifetime and identify a deliberately supplied dangling-reference example without executing undefined behavior.

1. Implement equivalent small functions taking a value, a const reference and a pointer with an explicit null policy.
2. Draw owners and borrowers for each function call. State what happens after a vector growth or owner destruction.
3. Implement a copied-value lookup and contrast it with a short-lived borrowed lookup.
4. Diagnose the supplied dangling example without executing it; submit the corrected version and reasoning.

Required cases: empty range -> absent; present ID -> copied expected position; missing ID -> absent; null optional target -> explicit result; copied result remains valid after source growth; dangling local identified by inspection.

Evidence to submit: Parameter-choice table, lifetime diagrams, null-policy tests and a repaired invalid-reference example.

Independent transfer: An event callback captures a local variable by reference and runs next frame. Show two safe designs and explain which one owns the captured state.

Assigned primary reading: R11: Interfaces, Resource Management and lifetime safety guidance.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does a raw pointer necessarily own its target?
2. Why is returning a reference to a local variable invalid?
3. Does const by itself make shared access thread-safe?
4. A `std::span` is copied into a job that runs after its vector is destroyed. Is the copy safe?
5. Why might returning a small position by value be better than returning a reference?

11.4 — Standard containers, iterators and error handling

Learning outcome: Use library abstractions while respecting invalidation and failure contracts.

Use the library, but learn the contract

A standard container packages allocation and element management behind a tested interface. It does not remove the need to understand its invariants. For `std::vector`, size is the number of constructed elements; capacity is the storage available before a growth allocation is needed. `reserve(100)` can increase capacity without creating 100 elements. `resize(100)` changes the element count.

A vector is a good default when the work needs contiguous storage, indexed access and frequent iteration. The teaching goal is not to replace the standard library with custom engine containers as early as possible. Implement educational containers later to understand costs, then justify any production replacement with measured requirements and a maintenance budget.

A range-based loop can read each element by const reference. A loop that copies each large object has different costs and possibly different behavior. Iterator categories also matter: an algorithm requiring random access cannot be assumed to work equally on a linked structure. State what operations the algorithm actually needs. [R11: SL; R13]

Invalidation is part of the interface

Vector reallocation invalidates pointers, references and iterators to its elements. Appending without reallocation preserves existing element references, but the past-the-end iterator changes. Erasing elements invalidates iterators and references at or after the erased position, even if no allocation occurs. Do not keep an iterator across a mutation just because capacity did not change.

Use the iterator returned by `erase` when removing elements during traversal, or use an appropriate erase algorithm. When entity references must survive structural changes, store an ID and re-query or design a stable-handle scheme. Reserving capacity is a useful allocation-control tool, not a general lifetime cure.

The lab deliberately forces growth and erasure, but it never dereferences a known-invalid pointer. Instead it validates results through fresh lookup and describes the invalidation event in the journal. This separates observing allocation behavior from performing an invalid memory access.

Expected absence versus an exceptional failure

A lookup that does not find a unit is often normal control flow. An optional result expresses this without a magic integer or a thrown exception. A malformed asset, an allocation failure and a violated internal invariant are different categories and may deserve different responses. Define the error taxonomy before scattering catches throughout the engine.

This bundle uses `std::optional` for expected absence and exceptions for selected failure paths. C++20 does not provide `std::expected` as a standard facility, so the examples do not depend on it. A production no-exceptions subsystem can use a result type, but must still preserve the same ownership and failure-state reasoning. Error transport does not replace an error contract.

Do not catch everything and continue as though loading succeeded. A catch block should recover, translate a failure at an appropriate boundary, add context, or terminate the operation cleanly. RAII in the next module keeps resources correct as an exception unwinds through the code. [R11: E]

Make mutation explicit and testable

The unit repository accepts unique IDs, appends units, finds by ID and removes by ID. A duplicate insertion returns a clear failure. A lookup returns a copied Unit in the teaching version, so external callers cannot keep stale element references. That choice also prevents callers from changing IDs behind the repository's invariant.

Separate a successful absence from a broken invariant. Removing a missing ID can return false without changing the container. Inserting an already-present ID should not silently create ambiguous lookup behavior. A test after every mutation verifies unique IDs and expected size.

This small API creates a path to the later resource manager: hide implementation details, publish ownership and error behavior, and test boundaries. The container implementation is important, but the observable semantics are what the game depends on.

Trace growth and removal without stale access

Start with an empty vector, reserve space, insert IDs 4 and 9, and query ID 4. The optional result is a value copy. Append enough distinct units to exceed the recorded capacity; do not assume a particular growth factor. Query ID 4 again and compare its value. Remove ID 4, then query it once more and expect absence.

For erase-during-iteration, trace a vector with hit points [0,5,0]. Erasing the first element shifts 5 and 0 toward the beginning. Incrementing an old iterator unconditionally can skip an element or use an invalid iterator. Assign the iterator returned by erase instead.

Neither `reserve` nor a check that an address looks unchanged grants permission to retain invalidated iterators.

Focus excerpt

The following excerpt isolates the idea; use the complete, tested program in `code/reference/lesson_11_4.cpp`. Pseudocode excerpts are not standalone C++ source.

```
for (auto it = units.begin(); it != units.end();) {
    if (it->hit_points == 0)
        it = units.erase(it);
    else
        ++it;
}
```

Common failure modes

Capacity is not size. A vector erase can invalidate references even without growth. A sentinel such as ID -1 can collide with the domain unless the domain excludes it. An empty catch block can turn a visible failure into corrupted game state.

Practical assignment

Original requirement: Port the unit list to std::vector and implement lookup with an explicit not-found result. Pass: test empty input, missing ID and growth; document which operations invalidate stored references.

1. Port the unit list to a vector-backed API with unique IDs and copied optional lookup results.
2. Test empty, present and missing queries, duplicate insertion, growth and erasure.
3. Write an invalidation table for reserve, append, erase and clear. Include the past-the-end iterator.
4. Classify normal absence, bad external input and invariant failure; explain the chosen error channel for each.

Required cases: empty lookup -> absent; duplicate ID -> rejected with unchanged size; growth preserves logical ID lookup; missing removal -> false and unchanged state; erase adjacent zero-HP units correctly; reserve changes capacity, not constructed count.

Evidence to submit: Unit repository, automated tests, error taxonomy and operation-specific invalidation table.

Independent transfer: A renderer stores vector iterators across an update that removes dead units. Redesign the frame boundary so rendering never uses invalid iterators.

Assigned primary reading: R11: standard-library use and error handling; R12: testing integration.

Check your understanding

Questions1–3 are retained from the original curriculum;4–5 are new application questions. Attempt all five before opening the key.

1. Does reserve(100) make a vector's size 100?
2. Why can a pointer to a vector element become invalid after growth?
3. What is wrong with ignoring every exception?
4. Can vector erase invalidate a reference when capacity stays constant?
5. When is an optional copied value a suitable lookup result?

Sources and provenance

Continuity with the original program

This expansion was prepared for the September28,2026 curriculum edition. The original program manifest identifies Volume03 as C++, Algorithms and Language Implementation, Modules11–15, GateC and400–650 planning hours. Its lesson IDs, titles, outcomes, prerequisites, practical assignments and assigned readings are preserved in `source/ORIGINAL_VOLUME_03_MANIFEST.json`. The 60 original quiz questions and answers were recovered from the original `CURRICULUM.html`; the expansion adds40 application questions without replacing the originals.

Teaching explanations, worked examples, code, additional exercises, reviews, bridge material and project elaborations are original instructional synthesis for this bundle. Primary reading references support the underlying concepts. No external textbook, paper, compiler distribution, proprietary SDK or font file is included. Linked resources retain their own terms. Current documentation was consulted on September28,2026; historical course archives are intentionally used for durable foundations. Not every section of every historical source was re-reviewed in full.

The original assignments are preserved even where the starter supplies only a first contract. The full lab remains a learner requirement. Complete reference examples provide comparison evidence, not pre-completed module projects. The source JSON distinguishes original questions from additions.

Primary reference directory

R06 — 6.042J Mathematics for Computer Science

MIT OpenCourseWare. Proofs, graphs, counting and discrete probability.

https://ocw.mit.edu/courses/6-042j-mathematics-for-computer-science-spring-2015/

R11 — C++ Core Guidelines

C++ Core Guidelines editors. Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

R12 — CMake tutorial

Kitware. Build systems; use documentation matching the installed release.

https://cmake.org/cmake/help/latest/guide/tutorial/index.html

R13 — 6.006 Introduction to Algorithms

MIT OpenCourseWare. Data structures, graph algorithms and dynamic programming.

https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/

R14 — Kaleidoscope and LLVM tutorials

LLVM Project. Compiler frontends, intermediate representation and JIT compilation.

https://llvm.org/docs/tutorial/

R15 — 6.004 Computation Structures

MIT OpenCourseWare. Logic, processor design, memory and systems.

https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/

R16 — Operating Systems: Three Easy Pieces

Remzi and Andrea Arpaci-Dusseau. Virtualization, concurrency and persistence; book by its authors.

https://pages.cs.wisc.edu/~remzi/OSTEP/

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University. Runtime systems and game-production programming.

https://graphics.cs.cmu.edu/courses/15-466/

P4-01 — CMake compile features and language standards

Kitware. Target requirements, language standards and extensions. Use the documentation matching your CMake release.

https://cmake.org/cmake/help/latest/manual/cmake-compile-features.7.html

P4-02 — What Every Computer Scientist Should Know About Floating-Point Arithmetic

David Goldberg / Oracle documentation. Author paper reproduced in official numerical-computation documentation; rounding and non-associativity background.

https://docs.oracle.com/cd/E19957-01/806-3568/ncg_goldberg.html

P4-03 — AddressSanitizer

LLVM / Clang. Compiler/runtime setup and kinds of executed memory errors detected. Detection is not an all-input proof.

https://clang.llvm.org/docs/AddressSanitizer.html

P4-04 — UndefinedBehaviorSanitizer

LLVM / Clang. Runtime instrumentation and supported checks. Platform/toolchain support must be verified.

https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

P4-05 — Language standards supported by GCC

GNU Compiler Collection. C/C++ standards and language modes. This course selects C17/C++20, not a newest-standard claim.

https://gcc.gnu.org/onlinedocs/gcc/Standards.html

P4-06 — Kaleidoscope: implementing a parser and AST

LLVM Project. Frontend architecture reading; the course language and integer semantics are independently specified and do not require LLVM libraries.

https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/LangImpl02.html

P4-07 — Fundamental types [basic.fundamental]

C++ working-draft editors / eel.is rendering. Current working-draft view, not the fixed published C++20 edition. Consult only the stable integer/representation concepts used here; later additions are outside this course baseline.

https://eel.is/c++draft/basic.fundamental

P4-08 — Vector modifiers, capacity and unordered-container requirements

C++ working-draft editors / eel.is rendering. Companion sections: https://eel.is/c++draft/vector.capacity and https://eel.is/c++draft/unord.req. Used for operation-specific invalidation rules; current draft contains later features not used by the course.

https://eel.is/c++draft/vector.modifiers

Reading by topic

Use R11 for ownership, interfaces, special members, exception safety and source-file organization; R12/P4-01 for builds; R13 for arrays, hashing, heaps, sorting, graphs and dynamic programming; R14/P4-06 for language frontends and compiler structure. R06 supports discrete proof and computability reasoning, and R15 connects representations to later hardware study. R16/R20 are cross-volume reading pointers, not dependencies of the supplied build.

Read the exact assignment printed under each lesson. Record the section actually studied and at least one exercise or derivation in the journal. The narrow course implementation deliberately differs from some reading examples: bounded integers instead of general floating-point language values, a small straight-line stack machine instead of an LLVM JIT, and mock resources instead of a graphics API. Do not merge incompatible assumptions silently.